Ders 12 Genişletilmiş Notu: Bağlam Değişimi, Süreç Yaratma ve Sonlandırma: fork, exec, wait, Zombi ve Öksüz Süreçler

Ders 12 Genişletilmiş Notu: Bağlam Değişimi, Süreç Yaratma ve Sonlandırma: fork, exec, wait, Zombi ve Öksüz Süreçler

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Zamanlayıcıların Tekrarı
- 2. Mobil Sistemlerde Çok Görevlilik
- 3. Bağlam Değişimi
- 4. Süreç Yaratma: fork ve exec
- 5. fork Sonrası İki Süreç ve fork'un Dönüş Değeri
- 6. Derste Gösterilen fork ve exec Örnekleri
- 7. exit ve return
- 8. Süreç Sonlandırma ve Düzgün Kapanış
- 9. Zombi ve Öksüz Süreçler, wait ile Toplama
- 10. Çok Süreçli Mimari: Google Chrome
- 11. Süreçler Arası İletişime Giriş
- Örnek Sorular
- Dersin Toparlaması

1. Zamanlayıcıların Tekrarı

Ders çevrim içi yapıldı. Asıl anlatım yaklaşık 00:10:30'da başladı. Ondan önce bir öğrenci `fork` komutunu tekrar anlatmasını istedi. Hoca, bugünkü konunun zaten süreç işlemleri (operations on processes) olduğunu, `fork`'un örnekleriyle ayrıntılı işleneceğini söyledi ve soruyu o bölüme bıraktı.

Hoca derse bir önceki derste konuşulan üç zamanlayıcıyı (scheduler) hatırlatarak başladı.

- **Kısa vadeli zamanlayıcı (short-term scheduler):** CPU'da bir sonraki adımda hangi sürecin çalışacağına karar verir.
- **Uzun vadeli zamanlayıcı (long-term scheduler):** İkincil bellekte (secondary storage) bekleyen ve çalıştırılması gereken işlerden hangilerinin belleğe alınacağına karar verir. Amaç, çoklu programlamayı olabildiğince verimli yapmaktır.
- **Orta vadeli zamanlayıcı (medium-term scheduler):** Bellekte duran ama şu an çalışması gerekmeyen süreçleri ikincil belleğe çıkarır (swap out), çalıştırılmak istenenleri belleğe alır (swap in). Böylece çoklu programlama (multiprogramming) daha verimli olur.

Uzun vadeli zamanlayıcının iki hedefi vardır. Bir yandan belleğe olabildiğince çok süreç almak ister. Öbür yandan bu süreçler arasında dengeli bir dağılım olmalıdır: hepsi CPU ağırlıklı (CPU bound) ya da hepsi G/Ç ağırlıklı (I/O bound) olmamalıdır. Böylece bazı süreçler G/Ç işini yaparken diğerleri CPU'da çalışabilir. Denge yoksa süreçler farklı kuyruklarda arka arkaya dizilir. Buna konvoy etkisi (convoy effect) denir.

2. Mobil Sistemlerde Çok Görevlilik

Hoca bu slaytın konunun tam yerinde olmadığını, ilerleyen bölümlerde daha çok konuşulacağını söyledi. Bilinmesi gereken temel nokta şudur: Mobil sistemler güç kısıtlı (power constrained) sistemlerdir. Sınırlı bir güç kaynağıyla çalışırlar. Akşam şarj edilen bir telefonun mümkünse iki gün dayanması beklenir. Bunun için enerjiyi dikkatli harcaması gerekir. Bu da güç tüketiminin işletim sistemi tasarımında bir kısıt, bir ölçüt olması demektir. Bu yüzden mobil işletim sistemlerinde farklı sınırlar ve farklı yapılar görülür.

iOS (eski sürümler): Süreçler çalışma biçimlerine göre ikiye ayrılır.

- **Ön plan (foreground) süreci:** Ekranda kullanıcıyla etkileşen süreç. Ön planda yalnız bir süreç etkin olabiliyordu.
- **Arka plan (background) süreçleri:** Arkada bir olayı bekleyen ya da bir iş yapan süreçler. Arka plan işi kısa ve periyodik olabilir. Örnek: Yeni e-posta var mı diye bakıp bir süre bekleyen süreç. Daha uzun soluklu da olabilir. Örnek: Arkada çalan müzik.

Eski iOS'ta arka plan süreçlerinin nasıl çalışacağını sistem belirliyordu. Sistemin sunduğu bazı arka plan hizmetleri arka planda çalışabiliyordu, kullanıcı süreçleri çalışamıyordu. Hoca bu durumun "yanlış hatırlamıyorsam iOS 4 ve öncesi" için geçerli olduğunu, sonradan yavaş yavaş açıldığını söyledi.

> **Ders sonrası not:** Sınırlı arka plan çok görevliliği büyük olasılıkla iOS 4 ile geldi. Yani kısıt iOS 4'ün kendisi için değil, iOS 4'ten önceki sürümler için geçerlidir.

Android: Böyle bir kısıt yoktur. Bir işin, kullanıcıyla etkileşim olmadığı dönemde arka planda çalışması gerekiyorsa bir servis (service) yazılır. Servis bağımsız bir süreçtir, ama uygulamanın yerine arkada gerekli işleri yapan bir temsilcisi gibi çalışır.

3. Bağlam Değişimi

3.1 Bağlam Nedir?

Bir sürecin yaşam döngüsü (new, ready, running, waiting vb. durumlar) daha önce kabaca konuşulmuştu. Süreç yaşamı boyunca adım adım ilerler: ilk komuttan (instruction) başlar, sonra bir sonraki komuta geçer. Bu takip program sayacı (program counter) ile yapılır. Program sayacı bir sonra çalışacak komutu gösterir. Ama o komut bir veri üzerinde çalışır. Bu veri yazmaç setindedir (register set). Bu yüzden yazmaç seti de önemlidir. Bunlara ek olarak sürecin o an etkin olarak kullandığı kaynaklar vardır.

Bunların hepsi o sürecin bağlamıdır (context). Çünkü süreç yaşadığı sürece dokunduğu, yaptığı, yapmakta olduğu ve yapacağı şeyleri gösterirler. Bağlam bilgisi süreç kontrol bloğunda (PCB, process control block) durur.

3.2 Bağlam Değişimi

Sık karşılaşılan durum şudur: Çalışan bir süreci işlemciden alıp kenara koymak, yerine başka bir süreç çalıştırmak, sonra ilk süreci geri koymak. Yani sürekli bir süreçten diğerine geçmek. Bunun için sürecin bütün bağlamının bir yere kaydedilmesi gerekir. İşin tamamına bağlam değişimi (context switch) denir:

1. Çalışan sürecin bağlamı kaydedilir.
2. Kenarda bekleyen başka bir sürecin bağlamı işlemciye yüklenir ve o süreç çalışmaya başlar.

3.3 Bağlam Değişimi Bir Ek Yüktür

Hoca "bağlam değişiminde ne yapmıyoruz?" diye sordu, sonra "çay demlemiyoruz gibi oldu" diye espri yaptı. Asıl nokta şudur: Bağlam değişimi yapmamızın amacı işlemciyi daha verimli kullanmaktır. Ama bağlam değişimi sırasında işlemcide kullanıcı uygulaması (user application) çalışmaz. Kaynağı kullanmasını istediğimiz uygulamaya kullandırmıyoruz. Onun yerine işletim sisteminin bir parçasını çalıştırıyoruz.

> **Hocanın vurgusu:** Bu açıdan bakınca bağlam değişimi sistem için bir ek yüktür (overhead). Önemli olan bu sürenin olabildiğince kısa olmasıdır.

Hocanın örneği: Normalde bir ya da iki birimde bitmesini beklediğiniz iş 20-30 birim sürerse işlemci zamanını boşa harcıyorsunuz. Oysa amaç CPU zamanını işlemler arasında etkin paylaştırmak ve anlamlı işleri çalıştırmaktır.

3.4 Süreyi Belirleyen Etkenler

- **İşletim sistemi tasarımı:** Bağlam değişiminde iki temel adım vardır: kaydetmek ve geri yüklemek. Süreç hakkında tutulan bilginin boyutu, tutulma biçimi, kaydedildiği yer ve kaydetme yöntemi bu işin süresini doğrudan etkiler.
- **Donanım desteği:** Donanımın bu iş için bir desteği olabilir ya da olmayabilir.

Hoca donanım desteğine SPARC makinelerini örnek verdi. Normalde bağlamda kaydedilmesi gerekenlerden biri yazmaç setidir. SPARC tasarımcıları işlemciye birden fazla yazmaç seti koymuştu. Hoca bunu Intel mimarisindeki AX, BX, CX, DX gibi yazmaçların birkaç kopyası olarak düşünmeyi önerdi ve "yanlış hatırlamıyorsam 32 kopya vardı" dedi. Tek bir komutla o an etkin yazmaç seti verilen yere kaydedilebiliyordu. Bu durumda 32 süreç arasında rahatça bağlam değişimi yapılabilir.

Bu başta güzel görünür. Ama günümüzde bir işletim sistemi 32 süreçle dönmez, çok daha fazlasına ihtiyaç vardır. 33. süreci yarattığınız andan itibaren bağlam değişimini yine bu donanım desteği olmadan yapmanız gerekir.

> **Ders sonrası not:** SPARC'ın birden çok yazmaç penceresi (register windows) asıl olarak fonksiyon çağrılarını hızlandırmak için tasarlanmıştır. Kopya sayısı işlemci modeline göre değişir; "32" sayısını hoca da kesin olmadan verdi. Ders kitabındaki genel fikir, bazı işlemcilerin birden fazla yazmaç seti sağlayarak bağlam değişimini hızlandırabileceğidir.

4. Süreç Yaratma: fork ve exec

Süreçlerle ilgili iki temel işlem konuşulacaktır: süreç yaratma ve süreç sonlandırma (termination). Hoca, dersin başında `fork` sorusunu soran öğrencinin sorusunun karşılığına bu bölümde gelindiğini söyledi.

4.1 İki Sistem Çağrısı

Süreç yaratma iki temel sistem çağrısıyla yapılır.

- **`fork`:** Çağıran sürecin birebir kopyası olan yeni bir süreç yaratır. Çağıran sürece ebeveyn (parent), yeni oluşan sürece çocuk (child) denir.
- **`exec`:** Diskteki bir programı yaratılmış sürecin içine yükler. Böylece süreç başka bir programa ait süreç haline gelir ve oradan çalışmaya devam eder.

Aynı sürecin birebir kopyası tek başına işe yaramaz. İşletim sisteminin açılışını hatırlayın: `init` süreci diskten belleğe kopyalanır ve başlatılır. Sonra sistem hizmetleri başlatılırken her seferinde önce bir `fork` yapılır. Ardından yaratılan yeni süreç farklılaştırılır: çalıştırılmak istenen program `exec` ile ona aktarılır.

4.2 Kaynak Paylaşımı

`fork` ile yaratılan iki süreç arasında ebeveyn-çocuk ilişkisi vardır. `fork` çağrısı biter bitmez iki süreç bütün kaynakları ortak kullanır. Sonrasında süreçler birbirinden farklılaştırılabilir. Böylece ortak kullanılan kaynaklar azalabilir ya da tamamen ortadan kalkabilir. Slayttaki kaynak paylaşımı seçenekleri (resource sharing options) üç tanedir:

- Ebeveyn ve çocuk bütün kaynakları paylaşır.
- Çocuk, ebeveynin kaynaklarının bir kısmını paylaşır.
- Ebeveyn ve çocuk hiç kaynak paylaşmaz.

Seçenekler her zaman ilkinden başlar: `fork` bittiği anda iki süreç aynı kaynaklara sahiptir. Kaynakların bundan sonra nasıl farklılaşacağı yazdığınız koda ya da sürece yüklediğiniz programa bağlıdır.

4.3 Çalışma Biçimleri

Hoca sordu: Kendinizin ikinci bir kopyasını niye yarattınız? Kalabalık görünmek için mi? Farklı bir iş yaptırmak için mi? Bir iş parçasını ona devredip sonucuna göre hayatınıza devam etmek için mi? Çalışma biçimi bu amaca göre belirlenir.

- **Ebeveyn bekler:** Ebeveyn çalışmasının bir noktasında çocuğu yaratır. Çocuk çalışır, biter, bir sonuç üretir. Ebeveyn bu sonucu bekler, sonra çalışmasına devam eder. Burada amaç genelde yapılacak işin yeni sürece devredilmesidir.
- **İkisi bağımsız çalışır:** Ebeveyn çocuğu yaratır. Çocuk kendi başına, ebeveyn de kendi başına çalışmaya devam eder. Hoca bunu "saldım çayıra, Mevlam kayıra metodolojisi" diye adlandırdı.

4.4 Kabuk Örneği

Ebeveynin beklediği çalışma biçiminin birebir örneği kabuktur (shell). Kabuk her seferinde sizden bir komut ister. Girilen komut için `fork` ile bir süreç yaratılır. Komuta karşılık gelen program `exec` ile bu çocuk sürece yüklenir. Sonuç ana sürece döner ve ekranda görünür. Sonra kabuk yeniden komut ister. Yani iş bir döngü içinde sürekli tekrarlanır.

4.5 Linux Süreç Ağacı

Slayttaki Linux süreç ağacında `init` süreci en üsttedir ve süreç kimliği (PID, process ID) 1'dir. Sistemdeki her sürecin tanımlanması için bir kimliğe ihtiyacı vardır. Sistem başlayınca ilk süreç olan `init` belleğe kopyalanır ve çalışmaya başlar.

`init` sonra çeşitli hizmetleri başlatır:

- `kthreadd`.
- `sshd`: güvenli kabuk (secure shell) bağlantısı için kullanılan arka plan süreci (daemon). `init` bunu başlatırken aslında kendinin ikinci bir kopyasını yaratır ve o kopyaya `sshd`'nin kodunu yükler.
- `login`: Sonra bir kabuk (`bash`) başlatır. Kabuktan önce `emacs` düzenleyicisi, sonra çalışan süreçlerin listesini almak için `ps` çalıştırılmıştır.
- `sshd` üzerinden de bir hizmet ve başka bir kabuk çalıştırılmıştır.

Ağaçta aşağı doğru inen her çizgi şu demektir: Süreç kendisinin bir kopyasını oluşturmuş (`fork`) ve çalıştırmak istediği programın kodunu oraya yüklemiştir (`exec`).

5. fork Sonrası İki Süreç ve fork'un Dönüş Değeri

5.1 Birebir Kopya, Ama İki Bilgi Farklı

`fork` ile yaratılan iki süreç arasında her şey aynıdır. Ama iki bilginin farklı olması gerekir.

- **Süreç kimliği (PID):** Kimlikleri aynı olan süreçleri birbirinden ayırt edemezsiniz. Her sürecin bir kimliği olmalı ve bu kimlik tek (unique) olmalıdır. Hoca bunun, dersin başında soru soran öğrencinin sorusunun cevabı olduğunu söyledi.
- **İkinci bilgi:** Hoca bunu öğrencilere sordu. Yazılan cevaplara şöyle karşılık verdi: Süreç kontrol bloğundaki bilgilerden, bağlamdan söz ediyoruz, bellek adresinden değil. Kaynak seçenekleri (resource options) gibi bir şey de değil. Çok daha basit, birbiriyle ilişkili iki şey var. Slayttaki süreç ağacı grafiğine bakınca kolayca görülür. Hoca öğrencilerden düşünmelerini istedi ve cevabı vermedi. Bu sorunun çözümü Örnek Sorular bölümündedir.

5.2 fork Ne Döndürür?

Hoca bunu yine süreç ağacı üzerinden anlattı. `init` ilk `fork` çağrısını yapıp `kthreadd`'yi oluşturdu. 2 numaralı süreci oluşturduğuna göre `init`'e dönen değer 2 olmalıdır. Yani ebeveyn,

oluşturduğu yeni sürecin kimliğini bilir. Aynı şekilde `sshd`'yi oluşturduğunda 3028 dönmesi gerekir.

Ama `fork`'tan dönüldüğü anda artık iki süreç vardır: ebeveyn ve çocuk. Çocuğa dönen değer 0'dır. Çünkü o çağrıyı aslında çocuk yapmadı; çocuk o çağrıdan sonra oluştu.

Dönüş değeri	Anlamı
`< 0`	Hata. Yeni süreç yaratılamadı; yalnız ebeveyn var
`0`	Çocuk süreçtesiniz
`> 0`	Ebeveyn süreçtesiniz; değer çocuğun PID'si

Böylece `fork`'un dönüş değerine bakarak ebeveyn ile çocuk ayrılır. Hoca burada Unix'teki bir alışkanlığı da hatırlattı: Düzgün tamamlanan bir program ya da başarılı bir fonksiyon genelde 0 döndürür. `fork`'ta ise 0, "çocuk süreçtesin" anlamına gelir.

> **Hocanın vurgusu:** `fork`'tan döndükten sonra önce ebeveynin mi çocuğun mu çalışacağı bilinmez. Hazır kuyruğunda (ready queue) aynı özelliklere sahip bir süreç daha vardır ve hangisinin çalışacağına zamanlayıcı karar verir.

5.3 Slayttaki C Programı

Slayttaki program, Unix'te `fork` ve `exec` sistem çağrılarının basit kullanımını gösterir. Kod şöyledir (`wait` için `sys/wait.h` başlığı ve çıktı satırı sonları derleme denemesinde eklendi):

```c
#include <sys/types.h>
#include <sys/wait.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid;

    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        execlp("/bin/ls", "ls", NULL);
    }
    else { /* parent process */
        /* parent will wait for the child to complete */
        wait(NULL);
        printf("Child Complete\n");
    }

    return 0;
}
```

Akış şöyledir:

- `pid = fork();` çağrısından döner dönmez iki süreç vardır ve ikisi de `pid` değerini kontrol eder.
- `pid < 0` ise hata vardır. Yeni süreç yaratılmamıştır, yalnız ebeveyn vardır. Bir hata mesajı yazılır.
- `pid == 0` ise çocuk süreçtesiniz. Bu örnekte çocuğa `ls` komutu `exec` ile yüklenip çalıştırılır.
- Aksi halde değer pozitiftir ve ebeveyn süreçtesiniz. Ebeveyn `wait` ile bekler, sonra ekrana mesaj yazar.

`wait` herhangi bir çocuğun sonlanmasını bekler. Bu programda zaten toplam iki süreç vardır. Karmaşık bir ağaç yoktur. Bu yüzden "şu PID'li sürecin bitmesini bekleyeceğim" demeye gerek yoktur.

Kod `gcc -Wall` ile uyarısız derlendi. Çalıştırıldığında önce `ls` çıktısı, ardından `Child Complete` yazıldı.

6. Derste Gösterilen fork ve exec Örnekleri

6.1 "before fork / after fork"

Bu örnek, bir öğrencinin "PID ne zaman sıfır oluyor?" sorusu üzerine anlatıldı (soru Örnek Sorular bölümünde). Hoca önce şunu vurguladı: `fork` ikinci, birebir aynı bir sürecin ortaya çıkmasını sağlar. Bu çağrı tamamlandıktan sonra olur.

Programda önce `printf` ile "Message before fork", sonra `fork`, sonra "Message after fork" yazdırılır. `fork` olmasaydı her iki mesaj birer kez yazılırdı. Ekran çıktısında ise "before fork" bir kez, "after fork" iki kez yazılmıştır. Nedeni şudur: `fork` tamamlandıktan sonra iki süreç vardır ve ikisi de birebir aynıdır. `fork`'tan döndükten sonra ikisi de bir sonraki komutu, yani `printf`'i çalıştırır.

Hoca bunun mantıklı bir program olmadığını, yalnız olayı kavratmak için basite indirgenmiş bir örnek olduğunu söyledi. Mantıklı olması için oluşturulan sürecin farklı bir şeye hizmet etmesi gerekir: başka bir program çalıştırmak ya da işin bir kısmını yapmak gibi. Bunun için de iki sürecin birbirinden ayırt edilebilmesi gerekir.

Derste gösterilen kodun yeniden yazılmış hali `gcc -Wall` ile derlendi:

```c
#include <stdio.h>
#include <unistd.h>

int main(void)
{
    printf("Message before fork\n");
    fflush(stdout);
    fork();
    printf("Message after fork\n");
    return 0;
}
```

Çıktı üç satırdır: bir "before", iki "after".

```
Message before fork
Message after fork
Message after fork
```

> **Ek bilgi (derste anlatılmadı):** Çıktı bir dosyaya ya da boruya (pipe) yönlendirildiğinde `printf` tamponu satır satır boşaltılmaz. `fork` anında tampondaki "before fork" yazısı da çocuğa kopyalanır ve iki kez yazılabilir. Yukarıdaki koddaki `fflush(stdout)` bunu önlemek içindir. Denemede `fflush` ile çıktı yönlendirilse de üç satır çıktı.

6.2 PID'leri Yazdırmak

Ebeveyn ile çocuğu ayırt etmek için süreç kimlikleri kullanılır. `getpid()` bir sürecin kendi kimliğini almak için kullanılan sistem çağrısıdır.

Bu örnekte ilk `printf` sürecin kendi kimliğini yazar. Sonra `fork` yapılır. Sonra her süreç hem kendi kimliğini hem `fork`'un dönüş değerini ("fork result") yazar. Yine ilk `printf`'in bir kez, sonrakinin iki kez çalışması beklenir.

Derste gösterilen ekran çıktısının okunuşu:

- İlk satır: Program derlenip çalıştırılınca süreç 2682 kimliğini almış.
- İkinci satır: "Process ID 2682, fork result 2683". İlk süreç `fork`'tan sonra çalışmaya devam etmiş. Ona dönen değer 2683, yani oluşturduğu çocuğun kimliği.

- Üçüncü satır: “Process ID 2683, fork result 0”. Bu çocuk süreç. Onun için `fork`’un dönüş değeri 0.

Hoca kimliklerin ardışık çıkmasının şans eseri olduğunu, sıralı gitmek zorunda olmadığını söyledi: “Kırk kere denesem bir kere denk geliyor, onda da denk gelmiş.”

Dönüş değerini kontrol ederek hangi süreçte olduğunuzu bilirsiniz. Bunun faydası iki süreci kaynak kod üzerinden farklılaştırmaktır (differentiation). En basit örnek kabuktur: Dönüş değeri 0 ise bu çocuk süreçtir; ona `exec` ile girilen komutu yükle, çalıştır, sonucu ekranda göster.

Aynı fikir kendi denememde `getppid()` de eklenerek denendi:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    printf("Process ID: %d\n", getpid());
    fflush(stdout);
    pid_t r = fork();
    printf("Process ID: %d, fork result: %d, parent ID: %d\n",
           getpid(), r, getppid());
    if (r > 0)
        wait(NULL);
    return 0;
}
```

Bir çalıştırmada ilk süreç 1142225 oldu. Ebeveyn satırında `fork` sonucu 1142226, çocuk satırında `fork` sonucu 0 çıktı. Çocuğun ebeveyn kimliği 1142225, yani ebeveynin PID’siydi. Bu çalıştırmada çocuğun satırı ebeveyninkinden önce yazıldı; bu da sıranın zamanlayıcıya bağlı olduğunu gösterir.

6.3 fork’u if İçinde Çağırmak ve İş Bölümü

Daha ilkel bir örnekte `fork` doğrudan bir `if` koşulunun içinde çağrılmıştır. Dönüş 0 ise (çocuk) `else` kısmına düşülür. Sıfırdan farklı bir değer, yani hata ya da yaratılmış bir çocuk varsa ebeveyn kısmından devam edilir.

Hoca bu şekilde farklılaştırmanın neye yarayacağını örneklerle anlattı:

- Bir süreç kameradan okuyup dosyaya yazar, öbürü yazılmış dosyayı alıp işler ve başka bir dosyaya yazar.
- Bir sözlükte ya da dosyada arama yapılacaksa beş süreç yaratılır, her biri farklı bir bölgede arar.
- Bir görüntü üzerinde filtreleme yapılacaksa her süreç görüntünün farklı bir alanı üzerinde çalışır.

6.4 exec Örneği

Sonraki slayttaki örnek, komut satırının ihtiyacı olan türden bir farklılaştırmadır: girilen komutu çalıştırmak için yeni bir süreç yaratmak, komutu o sürece yüklemek, sonucu göstermek ve yeni komut beklemek.

Kodda yine `fork` vardır. Ebeveyn kısmında özel bir şey yapılmaz. Çocuk kısmında `exec` ile `ls` komutu çalıştırılır. `ls` aslında dosya sisteminde duran bir programdır. Çağrıya `/bin/ls`, `ls`, `-l` ve `/etc/apache2/conf.d/` yolu verilmiştir. Yani bu yoldaki dosyaların listelenmesi istenmiştir.

Kod derlenmiş ve `./exec` ile çalıştırılmıştır. Ebeveyn kendi mesajını yazıp bitmiş. Çocuk “child runs an executable” benzeri mesajını yazmış. Sonra `ls` için “No such file or directory” hata mesajı çıkmıştır.

> **Hocanın vurgusu:** Bu hata `exec`’in çalışmadığı anlamına gelmez. `exec` çalışmış ve `ls`’yi sürecin içine yüklemiştir. Ama `ls`’nin `/etc/apache2/conf.d` klasörüne erişim izni yoktur ya da böyle bir klasör yoktur. Hatayı `ls` vermiştir.

Aynı yapı kendi denememde derlendi. Ebeveyn “Parent done” yazdı, çocuk mesajından sonra `ls: cannot access '/etc/apache2/conf.d/': No such file or directory` çıktı. Yani hatayı `exec` değil, yüklenen `ls` programı verdi.

Hoca bu örneğin henüz bir kabuk olmadığını belirtti. Kabuk olması için bir döngü içinde sürekli dışarıdan komut istenmesi, komutun ayrıştırılması (parse), parametrelerine ayrılması, her şey düzgünse bir çocuk süreç yaratılması ve komutun `exec` ile yüklenmesi gerekir. Bu örnek, bir kez çalışan çok ilkel bir kabuk gibidir. Geliştirilirse bir kabuğa dönüştürülebilir.

7. exit ve return

Hoca, `exit` ile `return`’ün aynı şey olmadığını, ama kullanım yeri bakımından aynı şeyi ifade ettiklerini söyledi. Konunun tam süreç sonlandırmaya denk geldiği için “süper oldu” dedi.

- **`return`:** C’deki anlamıyla içinde bulunduğunuz fonksiyondan çıkarır.
- **`exit`:** Süreci sonlandırır. `exit` dediğiniz anda sürecin sonlandırılmasını doğrudan başlatırsınız.

Slayttaki programın sonunda `return 0` vardır. Ama bu `return` `main` içindedir. `main`’den dönmek neredeyse `exit` gibidir, çünkü `main`’den döndükten sonra yapılacak şey yine sonlandırmadır. `return` başka bir fonksiyonun içinde yapılırsa aynı etkiyi göstermez.

`return`’ün `exit` benzeri etki gösterdiği bir başka yer iş parçacıklarıdır (thread). Bunlar ileride işlenecektir. Bir iş parçacığının çalışması için ona bir fonksiyon atanır. O fonksiyondan `return` edildiğinde iş parçacığı sonlanır. Bu, bir nevi `pthread_exit` çağırmak gibidir.

`exit`’in güzelliği şudur: Kodun herhangi bir noktasında çağırdığınızda, ana fonksiyonda mı, onun altındaki bir fonksiyonda mı, daha da altta mı olduğunuza bakmadan programı sonlandırırsınız. Hoca bunun bir sistem çağrısı olduğunu söyledi.

7.1 main’den -1 Döndürmek ve Hata Kavramı

Bir öğrenci `main`’den `return -1` yapmanın hata üretip üretmediğini sordu (soru Örnek Sorular bölümünde). Hocanın cevabının ana fikirleri:

- `return -1` de `exit` de hata üretmez. İkisi de yalnız sonlandırma durumudur.
- Unix sistemlerinde bir kabul vardır: Düzgün tamamlanan program 0 ile döner.
- Sistem çağrıları hata verdiğinde global bir değişkeni, `errno`’yu ayarlar. Hatanın ayrıntısına onun üzerinden bakılır. Bu konu iş parçacıklarında daha ayrıntılı konuşulacaktır.
- Kendi fonksiyonunuzdan ya da programınızdan -1, 1 ya da 0 ile dönmeniz doğrudan bir hata oluşturmaz. Bir şeyi kontrol edip bunun olmaması ya da yapılmaması gerektiğine karar verirsiniz, o yüzden dönersiniz.

Hocanın `exit` için verdiği örnek: Çok iş parçacıklı olmayan bir süreciniz var ve dinamik bellek ayırması gerekiyor. `malloc` bellek bulamadı ve hata döndü. Bu ayırma `main` içinde olmak zorunda değildir. İç içe çağrılmış fonksiyonların derinlerinde bir yerde olabilir. Her katmanda fonksiyondan dönüp “hata var mı” diye yukarıda tekrar tekrar kontrol etmek yerine, devam edemeyecek durumdaysanız orada `exit` çağırıp programı sonlandırmak mantıklıdır.

Öğrenci bunu “hata fırlatmak değil, sürecin başarılı olup olmadığına dair bir durum bilgisi göndermek” diye özetledi. Hoca “hadi hayırlısı” diyerek konuyu kapattı ve şunu ekledi:

> **Hocanın vurgusu:** İş parçacıklarında herhangi bir iş parçacığının içinden `exit` çağırırsanız yalnız o iş parçacığını değil bütün süreci yok edersiniz. Bu yüzden `pthread_exit` mi, normal `exit` mi çağırdığınız ayrıca önemlidir.

8. Süreç Sonlandırma ve Düzgün Kapanış

8.1 Sürecin Kendi Kendini Sonlandırması

Süreç yaratıldığı gibi ortadan da kaldırılır ve bu birkaç farklı yolla olabilir. İlki şudur: Süreç çalışır, işi biter ve kendini sonlandırır. Az önce görüldüğü gibi `main`’den `return 0` da sonunda sonlandırmaya götürür, `exit` de.

`exit` dendiği anda süreç etkin bir varlık olmaktan çıkar. Ama süreç durumlarını hatırlayın: Sonlanmış (terminated) diye bir durum vardır. Bu durumun nedeni şudur: Süreç bittiğinde ürettiği sonucun önemi olabilir. Bağımsız bir süreçse hiç umurunuzda olmayabilir. Ama birçok zaman sistem açısından o sürecin sonucu, düzgün sonlanıp sonlanmadığı önemlidir ve kontrol edilebilmelidir.

Bu kontrol için terminated durumunda işletim sistemi sürecin bellek imajını tutmaz. Ama sürecin temel bilgilerini ve dönüş değerini tutmaya devam eder. Bu kayıt, süreci yaratan süreç onu toplayana (reap) kadar orada kalır.

8.2 Ebeveynin Çocuğu Sonlandırması

Ebeveyn ile çocuk arasında doğrudan bir ilişki vardır. Hoca, ebeveynin `abort` sistem çağrısıyla çocuğu ortadan kaldırabileceğini söyledi.

Örnek: Bir işleme hattı (pipeline) kurdunuz. Bir süreç cihazdan görüntü alıp dosyaya yazar. Öbürü o dosyaya ön işleme (preprocessing) yapıp başka dosyaya aktarır. Bir diğeri onu işleyip başka bir şeye çevirir. Bütün bunları kontrol eden, kullanıcıyla etkileşimi sağlayan bir ana süreç vardır. Kullanıcı Ctrl+C'ye basıp "sistemi kapatacağım" derse ana süreç çocuk süreçleri tek tek kapatır.

Bunun dışında kullanılabilecek bir sistem çağrısı daha vardır: `kill`. Bu, sinyal göndermek için kullanılan yapılardan biridir ve sinyaller konusunda ayrıca konuşulacaktır.

> **Ders sonrası not:** Hoca "ebeveyn `abort` sistem çağrısıyla çocuğu sonlandırır" dedi. Ders kitabı burada genel bir "abort" işleminden söz eder. POSIX'te `abort()` fonksiyonu çağıran sürecin **kendisini** anormal biçimde sonlandırır (SIGABRT). Ebeveyn bir çocuğu sonlandırmak için `kill(pid, SIGTERM)` gibi bir sinyal gönderir.

8.3 Kullanıcının Sonlandırması ve Düzgün Kapanış

Süreç her zaman kendi isteğiyle sonlanmaz. Başka bir süreç ya da kullanıcı bunu başlatabilir. Örneğin çalışırken Ctrl+C ile, grafik arayüzde ise pencerenin çarpısına basılarak sonlandırılabilir.

Bu durumda sonlandırma sırasında her şeyin düzgün olduğundan emin olmak isteriz. Kapanış, etrafta bırakılan izleri ve etkileri en aza indirecek, kaynakları işletim sistemine düzgünce geri verecek şekilde yapılmalıdır. Buna düzgün kapanış (graceful shutdown) denir: "Beni sonlandıracaksınız, ama kapanırken etrafta bıraktıklarımı temizce kapatacağım ve düzgün bir durumda bırakacağım."

Temizlenmesi gerekebilecek şeyler:

- Çalışma durumunu belirleyen geçici dosyalar. Örneğin bir işlem (transaction) ya da oturum (session) mantığıyla çalışıyorsanız ara noktaları kaydettiğiniz, her çalışmada bakıp devam edip etmeyeceğinize karar verdiğiniz yapılar.
- Paylaşılan kaynaklarla ilişkili noktalar: oluşturulmuş ya da erişilen bir paylaşımlı bellek (shared memory) alanı, bir semafor üzerindeki kilit vb. Bunlar kaldırılmalıdır ki bu alanları kullanan diğer süreçler sorun yaşamasın.

8.4 atexit

Bunun için `atexit` kullanılır. `atexit` ile süreç sonlanırken çalışacak bir fonksiyon belirtilir. Kendi yazdığınız bir fonksiyonu kapanışta çalıştırılmak üzere kaydedersiniz: "Ben sonlanırken bu fonksiyon her zaman çalışsın." Fonksiyonun anlamı içine ne yazdığınıza bağlıdır. Slayttaki örnekte yalnız ekrana bir mesaj yazılıyordu. Gerçekte ise yukarıdaki kaynakları temizlemesi gerekir.

Bunu küçük bir programla denedim: `atexit` ile bir temizlik fonksiyonu kaydedildi, sonra iç içe çağrılmış bir fonksiyonun içinden `exit(3)` çağrıldı.

```
#include <stdio.h>
#include <stdlib.h>
```

```c
static void cleanup(void) { printf("cleanup called\n"); }
static void deep(void) { exit(3); }

int main(void)
{
    atexit(cleanup);
    deep();
    printf("never\n");
    return 0;
}
```

Çıktıda yalnız “cleanup called” yazıldı ve kabuktaki çıkış kodu 3 oldu. Yani `exit` derin bir fonksiyondan çağrılsa da süreci bitirdi ve kayıtlı fonksiyon çalıştı.

> **Ders sonrası not:** Altyazıda “add exit” diye geçen fonksiyonun adı `atexit`’tir. Hoca bunu “sistem çağrısı” diye andı; `atexit` aslında bir sistem çağrısı değil, C standart kütüphanesi fonksiyonudur.

9. Zombi ve Öksüz Süreçler, wait ile Toplama

9.1 Toplama (reap) ve Öksüz Çocuk

Her süreç sonlandığında terminated durumuna geçer. Çıkış değeri ve ürettiği sonuçla ilgili bilgi işletim sistemi tarafından saklanmaya devam eder. Bu, ebeveyn onu toplayana (reap) kadar sürer.

Ebeveyn ortada yoksa ya da bunu yapabilecek durumda değilse ne olur? Ebeveyn önce sonlanmış, çocuk sonradan sonlanmışsa, çocuğu toplayacak ya da sonucuna bakacak bir ebeveyn kalmamıştır. Böyle durumlarda işletim sistemi otomatik olarak devreye girer. Çocuğun ebeveynini `init` süreci olacak şekilde günceller. Toplama işlemini `init` yapar.

9.2 Zombi Örneği

Slayttaki örnekte `fork` yapılmış. Ebeveyn tarafı `while (1)` ile sonsuz döngüye sokulmuş. Çocuk tarafı bir mesaj yazıp doğrudan `exit` ediyor.

`fork`’tan dönünce iki süreç vardır ve hangisinin önce çalışacağı bilinmez. Ama bu senaryoda fark etmez. Ebeveyn sonsuz döngüye girdiği için orada kalır. Çocuğa ya baştan ya da daha sonra mutlaka sıra gelir. Çalışır çalışmaz mesajını yazıp çıkar.

Ekran görüntüsündeki `ps` çıktısı:

- 3380 numaralı süreç, adı `zombie`. Programın derlendikten sonraki çalıştırılabilir dosyasının adı da `zombie`. Ebeveyn `while` döngüsünde döndüğü için hâlâ çalışıyor görünür.
- Çocuk sistemde hâlâ vardır. Ebeveyn kimliği 3380, kendi kimliği 3381. Hoca bunların da ardışık çıkmasının tesadüf olduğunu tekrarladı.
- Çocuğun yanında `<defunct>` yazar. Yani artık işlevsel değildir. Sonlanmış ama toplanamamıştır, çünkü ebeveyn döngüde dönüp durmaktadır. Ortada etkin bir süreç yoktur, sürecin kalıntısı vardır. Bu kalıntı temizlenmediği için çocuk defunct durumunda kalır.

Kendi denememde ebeveyni sonsuz döngü yerine 2 saniye uyuttum. Bu sırada `ps` ile bakınca çocuk `Z` durumunda ve `[zombie] <defunct>` olarak göründü.

9.3 Öksüz Örneği

Tam tersi durumda ebeveyn `fork`’tan sonra çalışıp `exit` eder. Çocuk ise `while` döngüsünde kalır. Ebeveyn çalıştı, bitti, gitti. Çocuğun artık bir ebeveyni yoktur. Burada işletim sistemi devreye girer.

Slayttaki `zombie2` programında çocuğun kimliği 3467’dir ve ebeveyn kimliği 1 olmuştur. Yani çocuk `init`’e geçmiştir, çünkü ebeveyni çoktan çıkmıştır ve çocuğu kontrol edecek ya da sonucuna bakacak kimse kalmamıştır.

Ders sonrası not: Bugünkü Linux sistemlerinde öksüz süreç her zaman PID 1'e bağlanmayabilir. `systemd` kullanan sistemlerde bir "subreaper" süreç onu sahiplenebilir. Kendi denememde, ebeveyni hemen çıkan çocuğun yeni ebeveyni PID 1 değil, kullanıcı oturumunun `systemd --user` süreci oldu. Derste ve kitapta anlatılan klasik UNIX davranışıdır.

9.4 wait ile Beş Çocuğu Toplamak

Önceki slayttaki programda `wait`'e `NULL` parametresi verilmişti. Bu örnekte `wait`'e parametre verilip sonuç alınıyor. Hoca ayrıntısının ileride konuşulacağını söyledi.

Programın yapısı:

- `NUMBER_OF_CHILD` adlı bir makro 5 olarak tanımlanmış.
- Bir `for` döngüsünde her seferinde `fork` ile bir süreç yaratılıyor. Dönüş değeri bir dizinin elemanı olarak saklanıyor; yani yaratılan her çocuğun kimliği diziye yerleşiyor. Örnekte ebeveyn sırayla 3630, 3631, 3632, 3633, 3634 kimlikli çocukları yaratmış.
- `fork` dönüşü 0 ise (çocuk) doğrudan `exit` çağrılıyor. Çocuk hiçbir şey yapmadan çıkıyor. Çıkış kodu `exit(100 + i)` diye verilmiş, yani 100, 101, 102, 103, 104.
- Ebeveyn döngüde beş çocuk yarattıktan sonra `wait` çağırıyor. Amaç, çıkan çocuğu toplamak (reap) ve neyle çıktığını kontrol etmek.
- `wait`'e `child_status` değişkeninin adresi `&` ile veriliyor. Bilgi `wait` tarafından bu değişkene doldurulup geri geliyor. Ekranda çıkış kodları 100-104 olarak görünüyor.

Derste gösterilen kodun yeniden yazılmış hali `gcc -Wall` ile uyarısız derlendi:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUMBER_OF_CHILD 5

int main(void)
{
    pid_t pid[NUMBER_OF_CHILD];
    int i, child_status;

    for (i = 0; i < NUMBER_OF_CHILD; i++)
        if ((pid[i] = fork()) == 0)
            exit(100 + i);              /* çocuk hemen çıkar */

    for (i = 0; i < NUMBER_OF_CHILD; i++) {
        pid_t wpid = wait(&child_status);
        if (WIFEXITED(child_status))
            printf("Child %d terminated with exit status %d\n",
                   wpid, WEXITSTATUS(child_status));
    }
    return 0;
}
```

Denemede beş satır çıktı ve çıkış kodları 100'den 104'e kadar geldi.

9.5 Çıktı Sırası Neden Garanti Değil?

Hoca bu örnek üzerinden çalışma sırasını tartıştı. `fork`'tan dönünce ebeveynin mi çocuğun mu çalışacağı bilinmez; bu zamanlayıcının kararıdır. Hazır kuyruğunda aynı özelliklere sahip bir süreç daha vardır. Bu örnekte bilinen tek şey şudur: Çocuk çalıştığı an hemen çıkacaktır.

Yani "önce 3630, sonra 3631, sonra 3632…" diye sıralı çalışma garanti değildir. Belki ebeveyn çalıştı, bir çocuk yarattı, zamanlayıcı yine ebeveyni seçti, bir çocuk daha yarattı ve bu böyle sürdü. Sistemde beş çocuk birikti, sonra zamanlayıcı bunları canının çektiği sırayla çalıştırdı.

> **Hocanın vurgusu:** `wait`'te de bir sıralama yoktur. Birden fazla sonlanmış çocuk varsa `wait` bunlardan herhangi biriyle döner. Ekran görüntüsündeki sıra hem `fork` sonrası zamanlama kararından hem de `wait`'in bu davranışından kaynaklanmış olabilir.

10. Çok Süreçli Mimari: Google Chrome

Hoca, şimdiye kadar konuşulanların bir özeti olarak çok süreçli yapıyı neden istediğimizi topladı:

- Kabuk örneğinde olduğu gibi her komut için yeni bir süreç yaratıp çalıştırmak, sonra işe devam edebilmek.
- Yapılacak işi parçalara ayırıp daha verimli hale getirmek.

Örnek olarak Google Chrome'un benimsediği çok süreçli yapı gösterildi. Bu yapı yalnız üç süreçten oluşmaz. Slayttakiler, her sekme ya da sekme grubu için olabilecek süreç türleridir.

- **Tarayıcı (browser) süreci:** Kullanıcı arayüzünü sağlar.
- **Oluşturucu (renderer) süreci:** Web sayfasıyla iletişimi sağlar: HTML üzerinden bilgiyi alıp gönderir ve oluşan görüntüyü gösterir. Güvenlik için bir kum havuzu (sandbox) içinde çalıştırılır. Böylece yasa dışı ya da "yamuk" bir durum kontrol altına alınır.
- **Eklenti (plug-in) süreçleri:** Tarayıcılara farklı işlevler eklenebilir: akış (streaming), video yakalama (video capture) gibi. Bunların her biri ayrı bir süreç olarak çalıştırılır.

Bu ayrımın amacı yalıtımdır (isolation). Erişmeye çalıştığınız web sayfasında tarayıcınızı bozabilecek, normal dışı davranışa (abnormal behavior) yol açabilecek ya da bir güvenlik açığını tetikleyebilecek bir kod varsa, bunun etkisi oluşturucu süreçle sınırlanır. Eklentileri ayrı süreçlere koymak da aynı mantığa hizmet eder. Çalıştırdığınız eklentinin ne kadar güvenilir ya da hatasız (bug free) olduğunu bilemezsiniz. Biri diğerini bozabilir. Bunu en aza indirmek için her eklenti diğerlerinden yalıtılır.

11. Süreçler Arası İletişime Giriş

11.1 Bağımsız ve İşbirlikçi Süreçler

Kabuk tarzı bir yapı, çok süreçli bir program ya da bir tarayıcı yapın; eğer amacınız bir işi hızlandırmak, onu parçalara bölmek ya da bir işleme hattı kurmaksa, çoğu zaman işbirlikçi (cooperating) süreçlerden söz ediyorsunuz demektir.

- **Bağımsız (independent) süreçler:** Birbirinden tamamen bağımsızdır. Örnek: İşletim sisteminin açılışında oluşan süreçler.
- **İşbirlikçi süreçler:** Kabuk ve veri ya da iş üzerinden paralelleştirme yapan çok süreçli yapılar. Bunların bir şekilde etkileşmesi gerekir.

Etkileşim, süreçler arasında bir haberleşme mekanizması gerektirir. Bu da işletim sisteminin bu mekanizmaları sunması demektir.

11.2 İşletim Sisteminin Sorumluluğu

Hoca burada sık yapılan bir yanlış anlamayı en baştan düzeltmek istedi.

> **Hocanın vurgusu:** İşletim sistemi bu mekanizmaları yalnızca sunmaktan sorumludur. Onları yanlış ya da olmaması gereken şekilde kullanırsanız, kullanırken mantık ya da programlama hatalarınız varsa, işletim sisteminin bunu çözmesi ya da düzeltmesi pek mümkün değildir.

Örneğin senkronizasyon için sunulan mekanizmaları yanlış kullanırsanız süreçleri senkronize edemezsiniz. Bu açlık (starvation) ya da kilitlenme (deadlock) gibi durumlara yol açabilir. Sistemin kilitlenmeyi önleme ya da çözme özelliği varsa, sistemde oluşan kilitlenmeyi engeller ya da oluştuktan sonra çözmeye çalışır. Ama sizin çalışmanızı düzeltmez. Mantık hatanız sürdükçe sistem yine ve yeniden sorun yaşar. İşletim sistemi programlamadaki mantık hatanıza hiçbir şekilde karışmaz.

11.3 İki Temel Yöntem

Haberleşme için iki temel yöntem vardır: paylaşımlı bellek (shared memory) ve mesaj geçirme (message passing).

Paylaşımlı bellek: Süreçler fiziksel olarak var olan ve aralarında paylaşılan bir alana bilgi yazar ve okur. Bu yolla bilgi paylaşır, haberleşirler.

Burada dikkat edilmesi gereken nokta şudur: Aynı bellek alanına hem okuma hem yazma yönünde erişen birden fazla süreç vardır. Kimin nereye ne yazdığını kontrol etmek gerekir. En basit örnek üretici-tüketici (producer-consumer) problemidir. Hoca bunun büyük olasılıkla haftaya işleneceğini söyledi.

Hocanın örneği yine görüntü işleme hattıydı. Kameradan görüntü alıp kenara yazan süreç üreticidir (producer). O görüntüyü alıp işleyen süreç tüketicidir (consumer). Bunlar dosya yerine paylaşımlı bir bellek alanına yazıyor olsun. Üretici ilk kareyi koydu, tüketici aldı ve işledi. İkinci kareyi koydu, o da işlendi. Üçüncüyü koydu ama işleme uzun sürdü. Bu sırada üretici bir tane daha, bir tane daha koydu. Ne olacak? Alanın kapasitesine, yazma, okuma ve işleme hızlarına bağlı olarak veri kaybolabilir ya da verinin üzerine yazılabilir. Hızlı üreten bir üretici ile yavaş tüketen bir tüketici, ya da tersi durumunda sorun çıkar. Bunun için senkronizasyon mekanizmaları gerekecektir; bunlar ileride işlenecektir.

Mesaj geçirme: Bilgi mesajlara bölünür ve bu mesajlar süreçler arasında aktarılır.

Modern işletim sistemlerinde genelde her iki yöntem de gerçeklenir. Hangisinin kullanılacağı ihtiyaca göre seçilir.

Ölçüt	Mesaj geçirme	Paylaşımlı bellek
Dağıtık sistemler arası haberleşme	Daha mantıklı	Uygun değil
Aktarılacak veri	Küçük veride daha verimli olabilir	Büyük veride daha verimli olabilir
Sistem çağrısı	Her mesajın gönderilmesi ve alınması için	Yalnız alanı oluştururken
Hız	Her mesajda kullanıcı ve çekirdek kipi arasında geçiş var	Oluşturulduktan sonra doğrudan okunup yazılır

Mesaj geçirme çoğu zaman çekirdek (kernel) seviyesinde gerçeklenmiş bir yapıdır. Her mesajın aktarılması ve alınması için bir sistem çağrısı yapıp çekirdeğe geçmeniz, sonra kullanıcı kipine (user mode) dönmeniz, sonra yine çekirdeğe ve yine kullanıcı kipine geçmeniz gerekir. Yani sürekli bir geçiş işlemi vardır.

Paylaşımlı bellekte de alanın oluşturulması için sistem çağrıları gerekir. Ama oluşturulduktan sonra okuma ve yazma için hiçbir şeye gerek yoktur. Paylaşılan bellek alanı iki sürecin de adres uzayına (address space) eşlenmiştir (map). Hocanın deyişiyle oradan "takır takır" okunup yazılır.

11.4 MPI Hakkında

Ders sonunda bir öğrenci MPI'dan bahsedilip bahsedilmeyeceğini sordu. Hoca MPI'ın derste ayrıca anlatılmayacağını söyledi. MPI süreçler arası iletişimle ilgili konuşulabilecek şeylerden biridir, ama dersin doğrudan alanı değildir. Soket programlamada olduğu gibi, "aslında şöyle şöyle çalışıyor" düzeyinde genel olarak değinilecektir. Hocaya göre soket programlama ağ teknolojileri ve ileri ağ programlama derslerinde, MPI ise dağıtık sistemler (distributed systems) dersinde konuşulur.

Örnek Sorular

Bu derste ekrandan alınmış soru yoktur. Aşağıdaki sorular derste sözlü olarak sorulmuştur.

Soru 1 (derste sözlü, 00:32:44): `fork` ile yaratılan çocuk süreç ebeveynin birebir kopyasıdır. Ama iki bilginin farklı olması gerekir. Biri süreç kimliğidir (PID). Farklı olan ikinci bilgi nedir? İpucu: Süreç kontrol bloğundaki, bağlamdaki bir bilgi; bellek adresi ya da kaynak seçenekleri değil. Süreç ağacı grafiğine bakınca görülür. PID ile birbiriyle ilişkili.

> **Çözüm (derste verilmedi):** Farklı olan ikinci bilgi ebeveyn süreç kimliğidir (PPID, parent process ID). Çocuğun PPID'si, ebeveynin PID'sine eşittir; ebeveynin PPID'si ise kendi ebeveyninin kimliğidir. Süreç ağacındaki her çizgi bu ilişkiyi gösterir. Doğrulama: Bölüm 6.2'deki program `gcc -Wall` ile derlenip çalıştırıldı. Ebeveyn PID 1142225 iken çocuğun `getppid()` değeri 1142225 çıktı; ebeveynin `getppid()` değeri ise kabuğun kimliğiydi (1142159). Ayrıca `fork`'un dönüş değeri de iki süreçte farklıdır (ebeveynde çocuğun PID'si, çocukta 0).

Soru 2 (derste sözlü, 00:43:29): `fork` sonrası PID değişkeni ne zaman 0 oluyor?

Cevap: `fork` çağrısı tamamlandığı anda iki süreç vardır. `fork`'un dönüş değeri ebeveynde yeni yaratılan çocuğun kimliğidir (örnekte 2683). Çocukta ise 0'dır, çünkü çağrıyı çocuk yapmadı, çağrıdan sonra oluştu. Dersteki çıktıda "Process ID 2683, fork result 0" satırı bunu gösterir. Bu değere bakarak hangi süreçte olduğunuzu bilir, ebeveyn ile çocuğa farklı işler yaptırırsınız (Bölüm 6.1 ve 6.2).

Soru 3 (derste sözlü, 00:57:50): `main`'den `return` etmek `exit` kodlarına denk bir etki mi oluşturur? Örneğin `main`'den `return -1` dersem hata üretmiş olur muyum?

Cevap: Hata üretmiş olmazsın. `exit` ile de hata üretmezsin. İkisi de yalnız sonlandırma durumudur. Unix'te düzgün tamamlanan programın 0 ile dönmesi bir kabuldür. Sistem çağrıları hata verdiğinde ayrıntı `errno` global değişkenine yazılır. `main`'den dönmek neredeyse `exit` gibidir, çünkü ardından süreç sonlanır.

> **Ders sonrası not:** Çıkış durumu 8 bittir. `main`'den `return -1` eden bir program `gcc -Wall` ile derlenip çalıştırıldı; kabukta `$?` değeri 255 çıktı. 0'dan farklı olduğu için kabul gereği "başarısız" anlamına gelir.

Soru 4 (derste sözlü, 01:33:56): Derste MPI'dan bahsedilecek mi?

Cevap: Ayrıca anlatılmayacak. MPI süreçler arası iletişimle ilgili ama dersin doğrudan alanı değil; soket programlama gibi yalnız genel çalışma mantığıyla değinilecek (Bölüm 11.4).

Dersin Toparlaması

Bu derste önce zamanlayıcılar ve mobil sistemlerdeki çok görevlilik kısaca hatırlatıldı. Ardından bağlam değişimi işlendi: Sürecin bağlamı (program sayacı, yazmaç seti, kullandığı kaynaklar) PCB'de tutulur. Bağlam değişimi sırasında kullanıcı kodu çalışmadığı için bu bir ek yüktür ve süresi işletim sistemi tasarımına ve donanım desteğine bağlıdır.

Dersin ana kısmı süreç yaratma ve sonlandırmaydı. Süreçler `fork` ile kopyalanır, `exec` ile farklılaştırılır. `fork`'un dönüş değeri ebeveynde çocuğun PID'si, çocukta 0'dır ve bu sayede kaynak kodda iki süreç ayrılır. `fork` sonrası hangi sürecin önce çalışacağını zamanlayıcı belirler. Süreç `exit`, `main`'den `return`, ebeveyn ya da kullanıcı tarafından sonlandırılabilir. Düzgün kapanış için `atexit` ile temizlik fonksiyonu kaydedilir. Sonlanan çocuk, ebeveyn onu `wait` ile toplayana kadar zombi olarak kalır. Ebeveyni önce biten çocuk öksüz kalır ve `init`'e bağlanır. `wait` herhangi bir bitmiş çocukla döner. Son olarak Chrome'un çok süreçli mimarisi ve işbirlikçi süreçler için iki temel IPC yöntemi, paylaşımlı bellek ve mesaj geçirme tanıtıldı. Ders slayt 3.38'de bitti. Bir sonraki derste IPC'ye ve üretici-tüketici örneğine devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.